

***Universidad Nacional de General
Sarmiento***

***Segundo semestre 2026
Programación III – Comisión 1
Trabajo Práctico n°1
Threes!***

~~Integrantes del grupo:~~

Eberhardt Nicolás
Gauna Walter
Pogrebaj Marcos
Trillo Leonel

Grupo: 19

Docentes:

Bagnes Patricia
Bertaccini Daniel
Faccini Gonzalo

Introducción y Descripción del Juego

El presente trabajo práctico consiste en el desarrollo de una aplicación en Java para implementar el juego Threes!. El juego se desarrolla sobre una grilla de 4x4 casillas en la cual el usuario desplaza fichas numeradas. La regla de combinación inicial indica que los números 1 y 2 se fusionan para dar origen a un único 3, mientras que los valores superiores se combinan exclusivamente entre múltiplos de tres iguales (dos 3 forman un 6, dos 6 forman un 12, y así sucesivamente en escala exponencial).

Cada vez que el usuario efectúa un movimiento válido mediante las flechas del cursor o las teclas de dirección, todas las fichas se desplazan y, posteriormente, se inserta una nueva ficha de manera aleatoria (con valor 1, 2 o 3) en el borde de entrada correspondiente. La partida concluye de forma definitiva cuando el tablero se queda sin celdas vacías y no existen fichas contiguas habilitadas para combinarse. Al finalizar, la aplicación expone el puntaje acumulado por el usuario.

Metodología de Desarrollo

El proyecto fue desarrollado de manera colaborativa por el grupo de cuatro integrantes utilizando un repositorio compartido en GitHub. Esta herramienta de control de versiones permitió trabajar en paralelo sobre clases independientes y unificar los avances mediante commits y pushes frecuentes. Asimismo, se adoptó una política estricta de documentación interna mediante comentarios claros en el código fuente, facilitando la legibilidad y el mantenimiento compartido del sistema.

Arquitectura y Decisiones de Diseño

Para cumplir con el requerimiento de separar estrictamente la lógica de negocio de la interfaz gráfica, se adoptó el enfoque arquitectónico ***Forms and Controls*** descrito en la bibliografía del Moodle de la materia. En este modelo, la interfaz visual

(VentanaJuego) captura los eventos del usuario e invoca directamente los métodos públicos del modelo (Juego), manteniendo la regla fundamental de que el código de negocio desconoce por completo la existencia o tecnología de la interfaz.

La sincronización de la vista con el estado del modelo se resuelve mediante ***consulta activa (flow synchronization)***: tras cada acción ejecutada, la ventana le consulta explícitamente al modelo los valores actualizados (getValor, getPuntos, isCambio, getProximaFicha) para redibujar los componentes gráficos, evitando patrones más complejos de notificación como Observer.

El sistema fue modularizado en clases cohesivas y con bajo acoplamiento:

Tablero: Modela exclusivamente la grilla espacial de enteros y la matriz de cambios, resolviendo los desplazamientos y fusiones sin conocer detalles sobre puntajes o historiales.

Historial: Encapsula una estructura de tipo Pila (Stack) para gestionar los estados anteriores de la matriz, desacoplando a la clase principal de la colección utilizada.

Posicion y Direccion: Clases de datos y enumeraciones que reemplazan el uso de arreglos primitivos sueltos y cadenas de texto mágicas, otorgando mayor seguridad al tipado.

Juego: Actúa como el orquestador general de la partida, coordinando las reglas de negocio, el puntaje, las validaciones de fin de juego y la lógica de sugerencias.

GestorPuntajes: Administra la persistencia de los récords históricos en un archivo de texto (puntajes.txt), luego los muestra mediante una JTable en una ventana JDialog.

También nos tomamos la libertad de hacer una mejora de diseño en la parte de generación de fichas nuevas, el problema era que habían momentos en los que la aleatoriedad pura en la generación de dichas fichas podía llevar a rachas de 1s o rachas de 2s que cortaran el juego abruptamente. Implementamos un sistema inspirado en el Threes! original en el que se crea un mazo con 12 fichas (cuatro 1, cuatro 2 y cuatro 3) por medio de una ArrayList que luego se mezcla con ayuda de un random.

Ejemplos

El diseño aplicado prioriza el encapsulamiento y la robustez frente a entradas inválidas mediante el uso de buenas prácticas de programación orientada a objetos.

Como ejemplo de delegación de responsabilidades, el flujo de movimiento en la clase Juego centraliza la validación del desplazamiento:

```
Java
private boolean procesarMovimiento(Direccion
direccion) {

    tablero.limpiarCambios();

    historial.guardarEstado(tablero.copiarEstado());

    boolean huboDesplazamiento = false;

    if(direccion==Direccion.DERECHA)
        huboDesplazamiento = tablero.desplazarDerecha();
    elseif(direccion== Direccion.IZQUIERDA)
        huboDesplazamiento = tablero.desplazarIzquierda();
    elseif(direccion== Direccion.ARRIBA)
        huboDesplazamiento=tablero.desplazarArriba();
    elseif(direccion== Direccion.ABAJO)
        huboDesplazamiento=tablero.desplazarAbajo();

    if(huboDesplazamiento) {
        colocarFichaNueva(direccion);
        return true;
    }

    deshacer();
    return false;
}
```

Por otro lado, la clase Tablero encapsula el cálculo de los bordes de entrada para la aparición de nuevas fichas, aislando por completo esta lógica espacial de la vista:

Java

```
public List<Posicion>
getCeldasLibresEnBorde(Direccion
direccionMovimiento) {
    List<Posicion> libres = new ArrayList<>();
    int limite = tamano - 1;

    for (int i = 0; i < tamano; i++) {
        if (direccionMovimiento ==
Direccion.DERECHA && isVacio(i, 0)) {
            libres.add(new Posicion(i, 0));
        }
        if (direccionMovimiento ==
Direccion.IZQUIERDA && isVacio(i, limite)) {
            libres.add(new Posicion(i, limite));
        }
        if (direccionMovimiento == Direccion.ABAJO
&& isVacio(0, i)) {
            libres.add(new Posicion(0, i));
        }
        if (direccionMovimiento ==
Direccion.ARRIBA && isVacio(limite, i)) {
            libres.add(new Posicion(limite, i));
        }
    }
    return libres;
}
```

Adicionalmente, el código incorpora validaciones defensivas mediante el lanzamiento de excepciones (IllegalArgumentException) tanto en el control de dimensiones del tablero como en la verificación de rangos de coordenadas, previniendo estados inconsistentes en tiempo de ejecución.

El desarrollo de este trabajo práctico permitió afianzar los conceptos teóricos sobre separación de capas y diseño orientado a objetos. Como limitación técnica identificada a mejorar en futuras iteraciones, se destaca que el mecanismo de deshacer (*undo*) almacena copias completas de la matriz bidimensional en la pila del historial en lugar de calcular diferencias incrementales (*diffs*), un aspecto que consume mayor cantidad de memoria pero que prioriza la simplicidad y estabilidad en esta etapa del aprendizaje.

